

Task-Specific Training from a Text Corpus

A hands-on deep dive: building specialist models from documents and tickets — synthetic data, LoRA/QLoRA training, and honest evaluation, all on Kaggle and Colab

Recipient: Ian

Date: June 2026

Builds on: the fine-tuning curriculum you've already worked through

Format: a focused deep dive; do the runs on Kaggle / Colab as you go

1. Purpose

You've covered the breadth of fine-tuning already and done it well. This document goes deep on one specific, high-value workflow: turning a corpus of text (think support tickets plus documentation) into a small task-specific model that reliably performs a handful of recurring tasks. This is the workflow where synthetic data construction and disciplined evaluation matter most, and it's the area I want you genuinely strong in.

The emphasis throughout is hands-on, on free GPUs. Every part has something to run on Kaggle or Colab. Reading the theory is necessary but not sufficient; the understanding forms when you build a synthetic dataset yourself, train on it, and measure honestly whether it worked and whether anything else broke. Take your time. The goal is durable mastery of the corpus-to-specialist pipeline, not a fast result.

Treat this as a focused continuation of the curriculum. Where the curriculum asked you to run one of each kind of training, this asks you to run the full realistic loop end to end, with the data-construction and evaluation discipline that separates a specialist that actually works from one that looks fine on paper and fails in practice.

2. The scenario

Picture a realistic, common situation. A team handles a small number of recurring text tasks, say three or four: classifying an incoming item, extracting structured fields from it, drafting a grounded response, or routing it. You have a body of source material that already encodes how those tasks are done: past tickets, internal documentation, FAQs, resolved cases. The raw material exists, but it isn't in a form a model can train on, and there's nowhere near enough cleanly labeled examples to train a strong specialist directly.

The job is to go from that messy corpus to a small, reliable model for each recurring task. The path runs through synthetic data: you take what little verified material you have, expand it into a well-structured training set without inventing facts, train a compact LoRA specialist on it, and measure rigorously. This is exactly the workflow this document drills.

2.1 Fine-tuning, RAG, and RAFT for corpus-grounded tasks

Three ways to get domain knowledge into a system, and you should know when each applies, because corpus-grounded tasks are where the distinction bites:

- **RAG** retrieves relevant documents at inference time and puts them in the prompt. The weights don't change. Best when the knowledge changes often or is too large to bake in.
- **Fine-tuning** changes the weights so the model internalizes a task or style. Best for teaching a consistent task behavior (a fixed output format, a classification scheme, a way of reasoning) that you want reliably, without depending on retrieval each time.
- **RAFT (Retrieval-Augmented Fine-Tuning)** trains the model on (question, retrieved documents, answer) triples, deliberately including some irrelevant distractor documents, so it learns to use good context, ignore bad context, and stay grounded. It's the middle ground for corpus-grounded tasks: the model is fine-tuned to be good at the open-book setting it will actually run in.

For recurring tasks over a documentation corpus, the realistic answer is often a combination: a specialist fine-tuned for the task behavior, operating over retrieved context, and RAFT is the recipe that makes that robust. Read the RAFT paper to understand why training with distractor documents reduces hallucination: arxiv.org/abs/2403.10131.

3. The end-to-end pipeline (the mental model)

Hold this whole loop in your head before touching code. Every part below is one stage of it:

1. **Define the task precisely.** Exact input, exact desired output, exact success criterion. Vague tasks produce vague datasets.
2. **Build a small seed set from the corpus.** A handful of real, verified examples per task, in the exact input/output shape you defined.
3. **Expand with synthetic data.** Grow variety (and, for generation tasks, grounded coverage) from the seed without fabricating facts.
4. **Filter for quality.** Judge, dedup, decontaminate against the eval set, balance.
5. **Train a LoRA/QLoRA specialist.** Small model, the right recipe, on Kaggle or Colab.
6. **Evaluate on two axes.** Did it learn the task (gold set)? Did it get worse at anything else (regression guard)?
7. **Iterate.** Error-analyze, generate targeted data for the weak spots, retrain.

The two things people skip are step 4 (filtering and decontamination) and the regression half of step 6. Those are exactly where specialists silently go wrong, so this document spends the most time there.

4. Your working environment: Kaggle and Colab

Do all of this on free notebooks. No local servers needed. Both give you a free GPU and run every library this document uses.

Kaggle Notebooks

Free GPU (typically a T4, sometimes two; P100 at times), generous weekly GPU quota, and persistent datasets you can attach. Good for runs you want to repeat with the same data

attached. Enable the accelerator under Settings, and remember the weekly GPU-hour budget so you don't burn it on idle sessions.

Google Colab

Fastest possible start: open a notebook, set Runtime to a GPU, run. Free tier gives a GPU without guarantees on type or duration, so save checkpoints and outputs out of the session because it can disconnect. Almost every notebook linked below is built for Colab.

Practical notes that save hours

- Keep models and datasets small. The point is reps and understanding, not benchmark scores. A 0.5B to 3B model is plenty for everything here.
- On a small GPU, the usual OOM fixes: lower max sequence length, batch size 1 with gradient accumulation, gradient checkpointing, 4-bit (QLoRA), and bf16. You already know these from the curriculum; you'll lean on them here.
- Save every result out of the session (push the adapter and your metrics to a Kaggle dataset, Google Drive, or a repo). Free runtimes reset.
- When you generate synthetic data with a local model in the same notebook as training, unload the generator before training starts so they don't fight for VRAM (you already hit this in the curriculum).

5. Part 1 — Define the task and build seed data

A specialist is only as well-defined as its task. Before any data, write down: the exact input the model receives, the exact output you want (format included), and how you'll score correctness. For a classification task that's the label set and the metric; for extraction it's the target schema; for a grounded-response task it's what a faithful, correct answer looks like and how you'll judge it.

Questions to answer

- What is the precise input/output contract for each recurring task, and what is the success metric?
- How do you pull a small, verified seed set out of a messy corpus (tickets, docs) and put it in that exact shape?
- How many real seed examples do you realistically need before synthetic expansion is worth starting? (Often surprisingly few; see LIMA.)

Resource: LIMA, “Less Is More for Alignment” — evidence that a small set of high-quality examples goes a long way, which is the whole premise of seed-plus-synthetic:

arxiv.org/abs/2305.11206.

Hands-on

Pick one concrete recurring task and hand-build a small seed set in proper instruction/JSONL form, exactly as you did for the ticket classifier, but this time write down the input/output contract and metric first and keep a held-out gold set aside from the very start. That gold set is sacred: nothing derived from it ever enters training.

6. Part 2 — Synthetic data generation (the core)

This is the heart of the whole workflow and where most projects quietly fail. The goal is to turn a small seed into a larger, well-structured, diverse training set that teaches the task, without fabricating knowledge and without contaminating your evaluation. Get this right and a compact model can become genuinely good at the task; get it wrong and you train confident nonsense.

6.1 The core techniques

- **Paraphrase expansion** — generate many rewordings of an input while keeping the verified output. Teaches the model that many phrasings map to the same answer. The simplest, safest expansion, and the one you already did.
- **Self-Instruct** — use a strong model to generate new instruction/response examples from a seed of human-written ones, expanding coverage of the task space. Read: arxiv.org/abs/2212.10560.
- **Evol-Instruct (WizardLM)** — iteratively evolve seed examples along axes like added constraints, deeper reasoning, more concreteness, or broader scope, to produce harder and more varied training signal. Read: arxiv.org/abs/2304.12244.
- **Magpie** — extract instruction data directly from an aligned model with almost no seed, a useful trick when you have very little to start with. Read: arxiv.org/abs/2406.08464.
- **Document-grounded generation (corpus to instructions)** — the most relevant one here: turn raw documents into trainable instruction/QA pairs. Bonito does this as a task (unlabeled text in, instruction-tuning examples out), and Augmentoolkit is built specifically for converting unstructured documents into training data. This is how a documentation corpus becomes a task dataset.

6.2 Grounding: don't fabricate facts

The cardinal discipline. When you expand, you may vary phrasing, structure, difficulty, and persona freely, but the factual content of outputs must stay anchored to verified source material. For classification, keep the verified label. For grounded generation, generate the question and answer from an actual document and keep the answer faithful to it (this is exactly the RAFT construction: build examples from real passages, and include distractor passages so the model learns to ignore them). A synthetic set that invents plausible-sounding but wrong answers will train the model to be confidently wrong.

6.3 Quality filtering

Generation always produces some junk. Filter before training:

- **LLM-as-judge** — score each generated example for correctness, faithfulness, and adherence to the output contract; drop the weak ones. Understand how this is done and its biases (length bias, position bias). Read MT-Bench / LLM-as-judge: arxiv.org/abs/2306.05685.
- **Dedup and near-dedup** — remove exact and near-duplicate rows so the model isn't dominated by a few templates.

- **Decontamination** — the step people forget: make sure no generated row overlaps your eval/gold set or any public benchmark you'll measure on. The standard approach is n-gram overlap detection (MinHash with ~13-gram shingles). Do this before training, not after, or your eval is meaningless.
- **Balance and diversity check** — confirm the set isn't skewed (one label dominating, one phrasing style, one length). Skew produces a skewed model.

6.4 Mode collapse and teacher bias

A model trained purely on one teacher's synthetic output tends to inherit that teacher's quirks: its format habits, length distribution, refusal style. The established mitigations are to keep a real-data seed in the mix, optionally use more than one generator, and run the diversity check above. Know this failure mode by name so you can spot it in your own results.

6.5 Tools

Distilabel — framework for building synthetic-data pipelines (TextGeneration, EvolInstruct, UltraFeedback-style judging) with local vLLM or any OpenAI-compatible backend: distilabel.argilla.io.

Augmentoolkit — purpose-built for turning unstructured documents into training datasets, which is exactly the corpus-to-instructions step: github.com/e-p-armstrong/augmentoolkit.

Bonito — a model/task for generating instruction-tuning data from unlabeled text: github.com/BatsResearch/bonito.

Hands-on

On Kaggle/Colab, take your seed and expand it two ways: a simple paraphrase pass, and one Evol-Instruct-style pass that increases difficulty/variety. Run a quick LLM-as-judge filter, dedup, and decontaminate against your gold set. Keep the generator local in-notebook where you can (privacy-preserving workflow, the way you did before). You now have a real synthetic training set you built and understand line by line.

7. Part 3 — Train the specialist

Now train the LoRA/QLoRA specialist on your constructed set. The mechanics are the ones you already know; what matters here is applying them cleanly on the data you built. Use a small instruct model, match its chat template exactly, train to a genuinely low loss (remember your own lesson: “barely helped” was usually “undertrained”), and keep the run reproducible.

Questions to answer

- Which base model and why (size, license, instruct vs base, does it already emit your output format)?
- LoRA rank/alpha/target modules and learning rate for a task this size; when QLoRA is worth the quantization trade-off?
- How do you confirm the chat template is right so you don't get the free-completion bug you hit in module 1?

- Should you mix a small share of general-purpose data into training to protect against forgetting (see Part 4)?

Resources

Unsloth (free Kaggle/Colab LoRA/QLoRA notebooks, your main hands-on anchor):

github.com/unslothai/unsloth.

Hugging Face PEFT (LoRA/QLoRA) and TRL (SFT trainer): huggingface.co/docs/peft and huggingface.co/docs/trl.

LoRA paper: arxiv.org/abs/2106.09685 · QLoRA paper: arxiv.org/abs/2305.14314.

Hands-on

Train a LoRA on your synthetic set on Kaggle/Colab, to a low, stable loss. Save the adapter and your training config out of the session.

8. Part 4 — Evaluation done right (two axes)

This is the other place specialists silently fail, and the part of your question I most want you solid on. A fine-tune is only meaningful if you can prove two things at once: it got better at the target task, and it did not get meaningfully worse at anything else. Measure both, every time, against the same baseline before and after.

8.1 Axis 1 — task performance (did it learn?)

- Measure on the held-out gold set you reserved before any synthetic generation. Never on training data, never on anything derived from the gold set.
- Pick the metric that fits the task: accuracy / macro-F1 for classification, exact-match or field-level F1 for extraction, and for grounded generation a faithfulness / correctness judgment (often LLM-as-judge against the source passage), not just surface overlap.
- Always report base vs fine-tuned on the same gold set so the delta is real, the way you did on the ticket task.

8.2 Axis 2 — regression / drift (did anything break?)

Fine-tuning on a narrow task can quietly degrade general capability. This is catastrophic forgetting, and you only catch it if you measure for it deliberately. Establish a baseline on a small, fixed set of general benchmarks before training, then re-measure the fine-tuned model on exactly the same set. A large drop means the specialist gained its task at the cost of general ability, which is usually unacceptable.

- Use a small, fixed general benchmark slice as a sentinel (for an LLM, a manageable subset of something like MMLU; for an embedding model, an MTEB slice). The point is a consistent before/after number, not a full leaderboard run.
- Keep a couple of fixed general probes too (simple known-answer prompts) as a fast sanity check, the way you watched “capital of France” stay correct across the merge sweep.

- The standard mitigation if you see drift: mix a small share of general-purpose instruction data back into the training set, or merge the fine-tune with the base (you saw merging recover general ability and even regularize).
- Set acceptance criteria in advance: a minimum task gain and a maximum tolerated general regression. Decide before you train, and report honestly against them rather than rationalizing afterward.

Resources

Evaluation harnesses for the regression slice: EleutherAI lm-evaluation-harness github.com/EleutherAI/lm-evaluation-harness and Hugging Face lighteval github.com/huggingface/lighteval.

For embedding tasks, MTEB: github.com/embeddings-benchmark/mteb.

Hands-on

Before training, record your gold-set metric and a small general-benchmark number for the base model. After training, re-measure both on the identical sets. Write down all four numbers and state plainly whether your pre-set acceptance criteria were met.

9. Part 5 — Iterate

One pass is rarely the end. Error-analyze the gold-set misses, find the pattern (a confusable pair, an under-represented case, a format slip), generate targeted synthetic data for exactly that weakness, and retrain. This is the loop that turns a mediocre specialist into a good one, and it's far more effective than generating more undirected data. Stop when the task metric clears your bar and the regression stays inside tolerance, or when added data stops moving the gold-set number.

10. Established methods and papers (reading list)

The market-relevant foundations for this workflow. Read the abstracts and skim the methods; you don't need every detail, but you should know what each contributes and when to reach for it.

- **LoRA** — low-rank adapters: arxiv.org/abs/2106.09685
- **QLoRA** — 4-bit base + adapters: arxiv.org/abs/2305.14314
- **Self-Instruct** — seed-to-dataset expansion: arxiv.org/abs/2212.10560
- **Evol-Instruct / WizardLM** — evolving difficulty/variety: arxiv.org/abs/2304.12244
- **LIMA** — small high-quality sets go far: arxiv.org/abs/2305.11206
- **Magpie** — near-seedless instruction extraction: arxiv.org/abs/2406.08464
- **RAFT** — retrieval-augmented fine-tuning for corpus-grounded tasks: arxiv.org/abs/2403.10131
- **MT-Bench / LLM-as-judge** — model-based evaluation and its biases: arxiv.org/abs/2306.05685

11. Hands-on exercise (the real thing)

Two tracks. Do Track A first as a clean, measurable warm-up, then Track B, which mirrors the corpus-to-specialist scenario most directly. Small models, small data, all on Kaggle/Colab.

11.1 Track A — a measurable classification specialist

Use *banking77* (*PolyAI/banking77* on Hugging Face), 77 fine-grained, deliberately confusable intents. Because the intents are easy to confuse, a base model struggles, so there is real headroom for both fine-tuning and synthetic data to show a measurable effect.

8. Define the contract (query in, one of N intents out) and the metric (accuracy / macro-F1). Reserve a held-out gold set.
9. Measure the base model on the gold set, and record a small general-benchmark sentinel number.
10. Build a small seed, then expand it with a paraphrase pass and an Evol-Instruct-style pass; judge-filter, dedup, decontaminate against the gold set.
11. Train a LoRA. Then measure the gold-set metric and the sentinel again.
12. Compare three conditions: base, seed-only fine-tune, seed+synthetic fine-tune, so you can isolate exactly what the synthetic data contributed. Report which confusable intents it helped most.

11.2 Track B — a corpus-grounded specialist

Pick a small public documentation or FAQ corpus (any product's docs, a help-center export, or a public docs/QA dataset). This track is about turning documents into a task dataset and training a grounded specialist.

13. Choose one grounded task (for example: answer a question from the docs in a fixed format, or extract structured fields from a document).
14. Turn the corpus into instruction/QA training examples using a document-grounded approach (Augmentoolkit or Bonito, or a Distilabel pipeline), generating questions and faithful answers from real passages. For a RAFT-style version, include distractor passages so the model learns to ignore irrelevant context.
15. Judge-filter for faithfulness to the source, dedup, decontaminate against your gold set.
16. Train a LoRA specialist, then evaluate: faithfulness/correctness on the gold set (LLM-as-judge against the source), plus the regression sentinel before/after.
17. Iterate once: error-analyze the misses, generate targeted data for the weak spot, retrain, re-measure.

12. What to produce

When you've worked through it, send me a write-up, kept practical. For each track: the task contract and metric, how you built the seed and the synthetic set (techniques, filtering,

decontamination), the base vs seed-only vs seed+synthetic numbers on the gold set, the regression sentinel before/after, and whether your pre-set acceptance criteria were met. Note what broke and what surprised you, the way your existing log already does well.

Then a short paragraph stepping back: given a messy corpus and a recurring task, how would you design the synthetic-data construction and the evaluation so a small specialist comes out reliable and doesn't regress. That's the understanding this whole document is meant to build, and it's what I'll want you applying for real next.

No rush on any of it. Depth over speed. Use Claude freely to explain concepts, design prompts for the synthetic generation, and debug the notebooks as you go, and keep doing the runs on Kaggle and Colab.